



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES

■ BackPropagation

- TLY1102

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern lighting. In the foreground, there is a desk with a coffee cup and some papers. A blue square with the number '01' is overlaid on the left side of the image.

01

Introducción

• Introducción

Hemos incorporados dos conceptos nuevos:

- ❑ Función de error
- ❑ Descenso del gradiente

Como la base de los algoritmos de optimización de una red neuronal.

Ahora, nos enfocaremos en uno de los algoritmos más utilizados en redes neuronales profundas:
BACKPROPAGATION.

• Introducción

¿Para qué sirve **BACKPROPAGATION**?

El **descenso del gradiente intenta optimizar la función de error**, buscando cómo influye el cambio de un parámetro en el error final de la red.

Recordemos que el descenso del gradiente es un algoritmo caro computacionalmente hablando, que hoy en día se ocupan algoritmos basados en descenso del gradiente como Adam, Rmsprop y en algunos casos SGD (descenso del gradiente estocástico).

Estas versiones de algoritmos de optimización son menos sensibles al valor λ .

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)}; y^{(i)}) \quad \Rightarrow$$

$$\hat{W}^i = \hat{W}^i - \lambda \frac{d\mathcal{L}}{dW^i} \Big|_{\hat{\theta}}$$

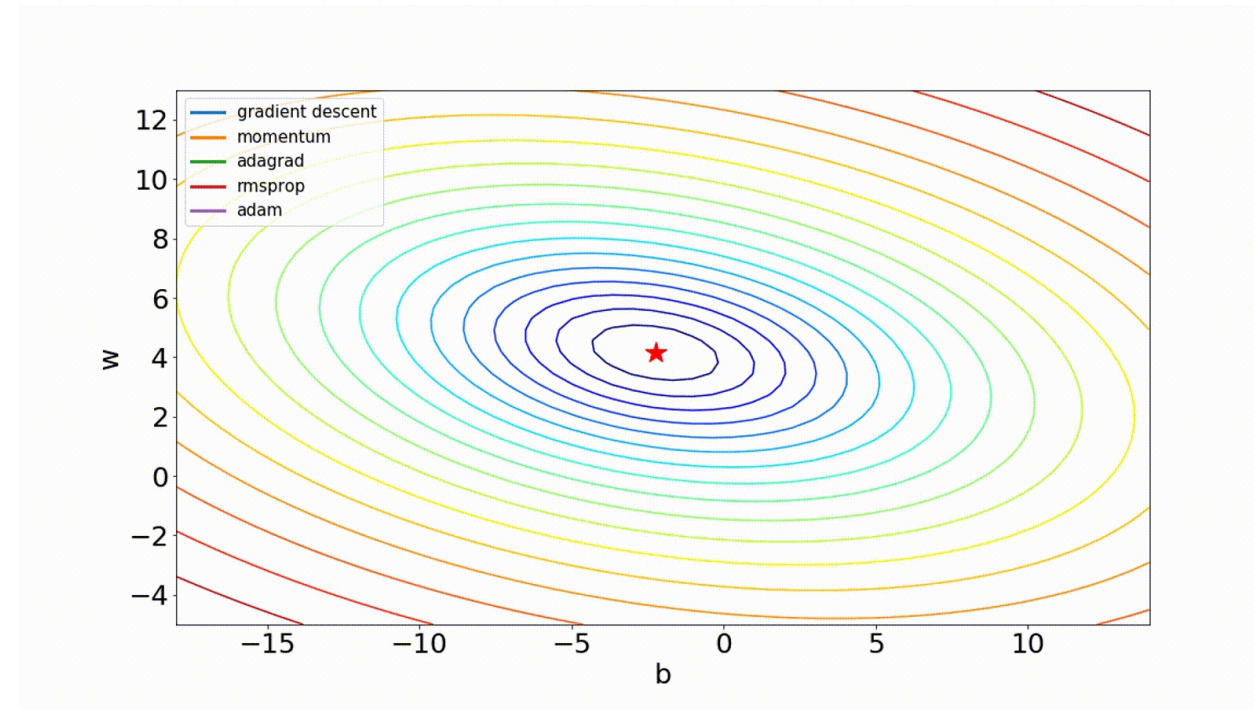
• Introducción

¿Para qué sirve **BACKPROPAGATION**?

Estos algoritmos intentan encontrar **el mejor valor de λ o la mejor manera de actualizarlo**.

Es decir, si el parámetro $W_{i,j}$ cambia, el descenso del gradiente verá cómo impacta ese cambio en las neuronas de la capa que lo recibe y cómo impacta en las capas sucesivas y finalmente en la función de error.

Recordemos que **esta función entrega un número real**, un escalar, que va variando de acuerdo a cómo voy actualizando los valores de los parámetros y el objetivo es que este número sea lo más chico que pueda.



• Introducción

¿Para qué sirve **BACKPROPAGATION**?

Finalmente, para ver cómo influye el cambio de ese parámetro en la función final, **se hace derivando la función de error con respecto al parámetro ($W_{i,j}$) que cambiamos.**

Para efectos pedagógicos, simulamos que nuestra función de error tenía solo un parámetro, pero recordemos que, en la realidad, son millones e incluso billones. Por lo tanto, mover los billones de parámetros **no es una tarea sencilla ni barata desde el punto de vista de cómputo y tiempo.** Derivar pasa a ser una tarea compleja.

Y acá aparece en acción el algoritmo de **backpropagation** (década de los 80), que permite de manera eficiente, realizar este cálculo.

NOTA: *En la práctica, en general, nadie programa este algoritmo, las librerías lo traen incorporado, sin embargo, es muy importante conocer su funcionamiento.*

• Introducción

¿Para qué sirve **BACKPROPAGATION**?

BACKPROPAGATION, traducido al español como “propagación hacia atrás de errores”, es un método para calcular de manera eficiente el gradiente (pendiente) de la función de pérdida en relación con todos los pesos de la red.

Consta de dos fases principales:

1. **Una pasada hacia adelante**, donde los datos de entrada pasan a través de la red para generar una salida.
2. **Una pasada hacia atrás**, donde **la salida se compara con el valor objetivo y el error se propaga de nuevo a través de la red para actualizar los pesos.**

La esencia de **BACKPROPAGATION**, es la regla de la cadena del Cálculo (lo veremos en la siguiente sección), que se utiliza para calcular los gradientes de la función de pérdida para cada peso multiplicando los gradientes de las capas detrás de él. Este proceso revela cuánto contribuye cada peso al error, lo que proporciona un camino claro para su ajuste.

• Introducción

¿Para qué sirve **BACKPROPAGATION**?

Comprender **backpropagation**, puede ser un poco complicado al comienzo, por lo cual es importante dejar en claro la diferencia que existe con el algoritmo de optimización, en nuestro caso: el descenso del gradiente.

Descenso del gradiente es un algoritmo que permite ajustar los parámetros (pesos y bias) y para eso necesita usar la derivada parcial y para calcular derivadas, se utiliza el algoritmo de backpropagation, el cual permite realizar esta tarea de manera eficiente.

Y para cerrar lo anterior, el ajuste repetitivo de los parámetros es lo que conocemos como **entrenamiento**.



02

Funcionamiento

1.- **Backpropagation** hace un uso intensivo de la regla de la cadena para calcular las derivadas parciales.

2.- **Backpropagation** puede ser utilizado en otras aplicaciones, en redes neuronales es uno de los usos que se le da.

En general, **backpropagation** es un algoritmo matemático para calcular derivadas de manera eficiente.

$$y = f(u)$$

$$u = f(x)$$

$$y = f(f(x))$$

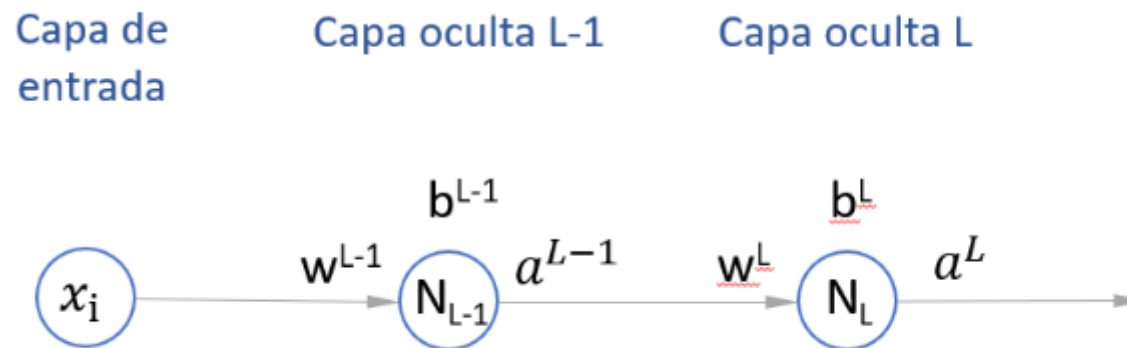
$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$$

La regla de la cadena es una norma de la derivación que nos dice que: teniendo una variable “y” que depende de “u”, la cual depende a la variable “x”, entonces...

la razón de cambio de “y” respecto a “x” puede estimarse como el producto de la derivada de “y” con respecto a “u” por la derivada de “u” respecto a “x”.

Vamos a llamar a la segunda capa oculta (la más próxima a la salida de la red) L y a la primera capa, $L - 1$.

El comportamiento de la neurona artificial de la capa L (la neurona N_L) viene determinado por el peso del enlace que la une con el valor devuelto por la capa anterior a $L - 1$, w^L , y por el bias a añadir, b^L .



En las siguientes expresiones, “ C ” es la función de coste resultante de analizar el comportamiento de la red neuronal con todas las muestras del conjunto de entrenamiento.

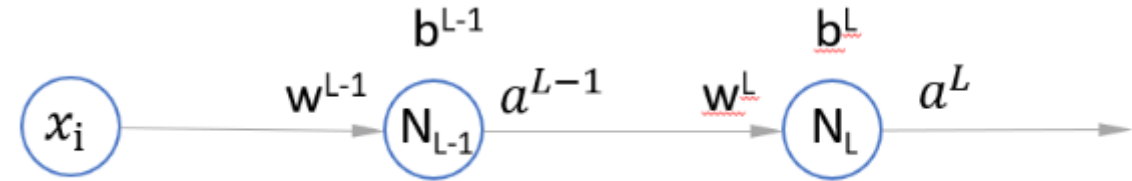
La derivada parcial de la función de coste con respecto a estos dos parámetros, w^L y b^L :

$$\frac{\partial C}{\partial w^L} \quad \frac{\partial C}{\partial b^L}$$

Capa de
entrada

Capa oculta L-1

Capa oculta L



Para una de las muestras, la i -ésima, las expresiones anteriores vendrían dadas por:

Derivadas parciales de una muestra en concreto:

$$\frac{\partial C_i}{\partial w^L} \quad \frac{\partial C_i}{\partial b^L}$$

En la siguiente expresión, " y " es el valor que debería devolver la red para la muestra i -ésima (es decir, la etiqueta asociada a dicha muestra) y " $\hat{y}$ " es el valor que en realidad devuelve la red para la muestra en cuestión.

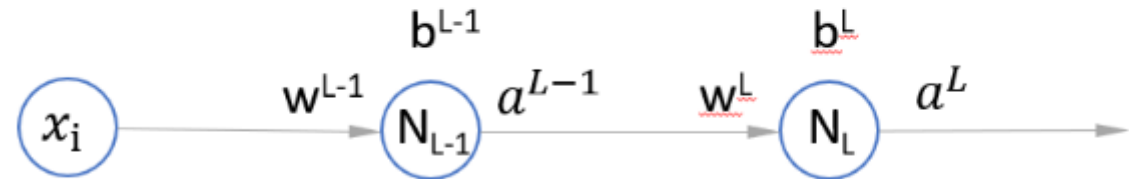
Este valor viene dado, como ya sabemos, por la multiplicación del valor devuelto por la neurona anterior por peso del enlace, más el bias, y pasando el resultado de esta combinación lineal por la función de activación σ .

$$\hat{y} = \sigma(w^L a^{L-1} + b^L)$$

Capa de
entrada

Capa oculta L-1

Capa oculta L



La **función de coste** puede expresarse como:

$$C_i = (y - \sigma(w^L a^{L-1} + b^L))^2$$

Por comodidad, llamemos z^L a la combinación lineal $w^L a^{L-1} + b^L$:

$$z^L = w^L a^{L-1} + b^L$$

De forma que:

$$\hat{y} = \sigma(z^L)$$

Pues bien, el cálculo nos dice que la derivada de la función C_i (El error) con respecto a w^L (es decir, cómo varía C_i cuando variamos w^L) coincide con la derivada de C_i con respecto a $\sigma(\hat{y})$, multiplicado por la derivada de σ respecto de z^L , multiplicado por la derivada de z^L con respecto a w^L . Esto es lo que se llama **regla de la cadena**:

$$\frac{\partial C_i}{\partial w^L} = \frac{\partial C_i}{\partial \sigma} \frac{\partial \sigma}{\partial z^L} \frac{\partial z^L}{\partial w^L}$$

La buena noticia es que todas estas derivadas parciales son fácilmente calculables:

- La derivada parcial de C_i con respecto a σ es, para la función de coste escogida, la derivada de $C_i = (y - \sigma(w^L a^{L-1} + b^L))^2$ es $2(y - \sigma)$.
- La derivada parcial de σ con respecto a z^L es la derivada de la función de activación con la que estemos trabajando.
- La derivada parcial de z^L con respecto a w^L es $L - 1$.

De modo análogo, la derivada parcial de C_i con respecto a b^L vendría dada por una expresión semejante a la anterior:

$$\frac{\partial C_i}{\partial w^L} = \frac{\partial C_i}{\partial \sigma} \frac{\partial \sigma}{\partial z^L} \frac{\partial z^L}{\partial b^L}$$

...lo que resulta, incluso, más fácil de calcular pues la derivada parcial de z^L con respecto a b^L es 1 (además de las derivadas ya calculadas en la expresión).



03

Ejemplo

Ejemplo

El DESCENDIENTE DEL GRADIENTE es el algoritmo de optimización que usamos para minimizar la función de pérdida (**Lost function**).

Para lograr esto el algoritmo cambia los parámetros W y b hasta encontrar los valores que minimicen a la función. Por ejemplo, este cambio se realiza con:

```
W = W - learning_rate * dW
```

Como podemos observar, **necesitamos la derivada del parámetro W** , que es dW , y un **hiperparámetro** llamado **learning rate**.

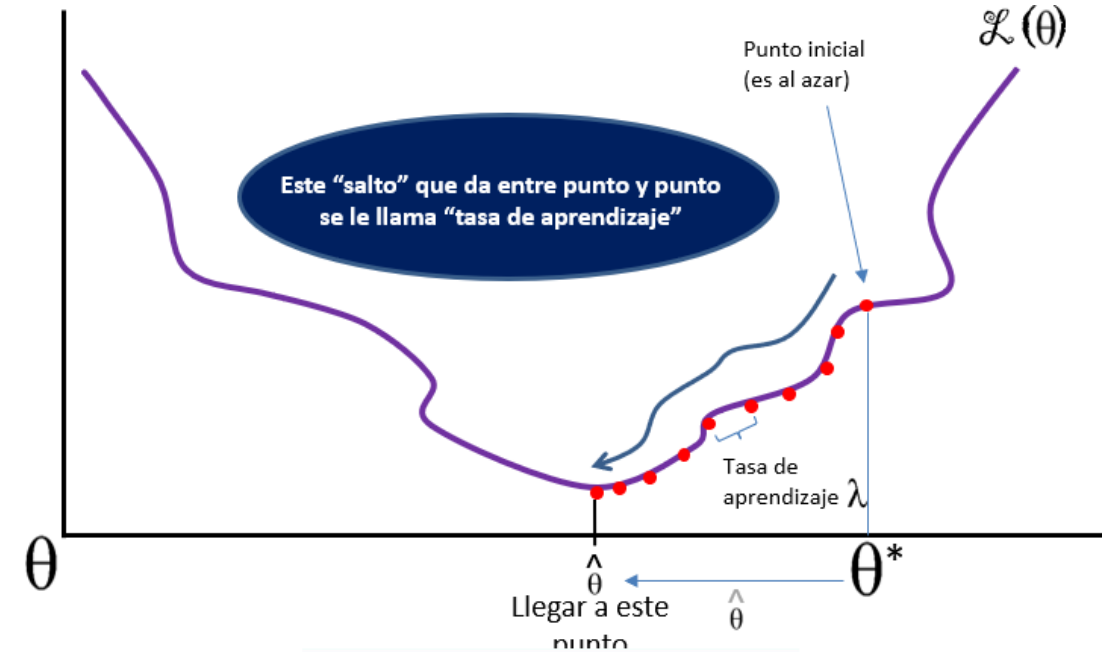
El **learning rate** (**Tasa de Aprendizaje**) podemos describirlo como la cantidad que W aumentará o disminuirá y la **derivada** podemos decir que nos da la respuesta de como cambiando el valor de W se afecta al resultado de la función de pérdida.

Recordemos :

Función de Pérdida (Loss)

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)}; y^{(i)})$$

Ejemplo



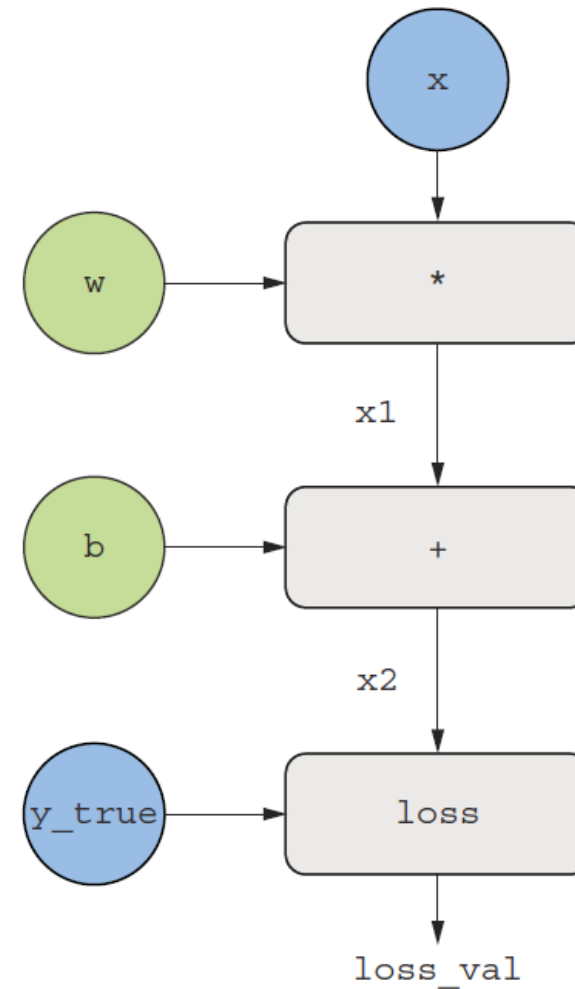
- El algoritmo comienza en un punto al azar (el valor inicial de W_0) dentro de la función.
- El objetivo es iterar, hasta llegar al punto que minimiza la función de pérdida.
- En el paso a paso, cada vez que ajusto o cambio el W , se calcula la derivada en este punto.

(Derivar significa encontrar la pendiente de la función en ese punto).

Ejemplo

Para obtener las derivadas de W y de b tenemos que realizar una serie de pasos, a esto se le llama ***backpropagation***.

Para ilustrar esto, supongamos que nuestra red está compuesta de una capa/neurona y todas las variables a usar son escalares.



Ejemplo

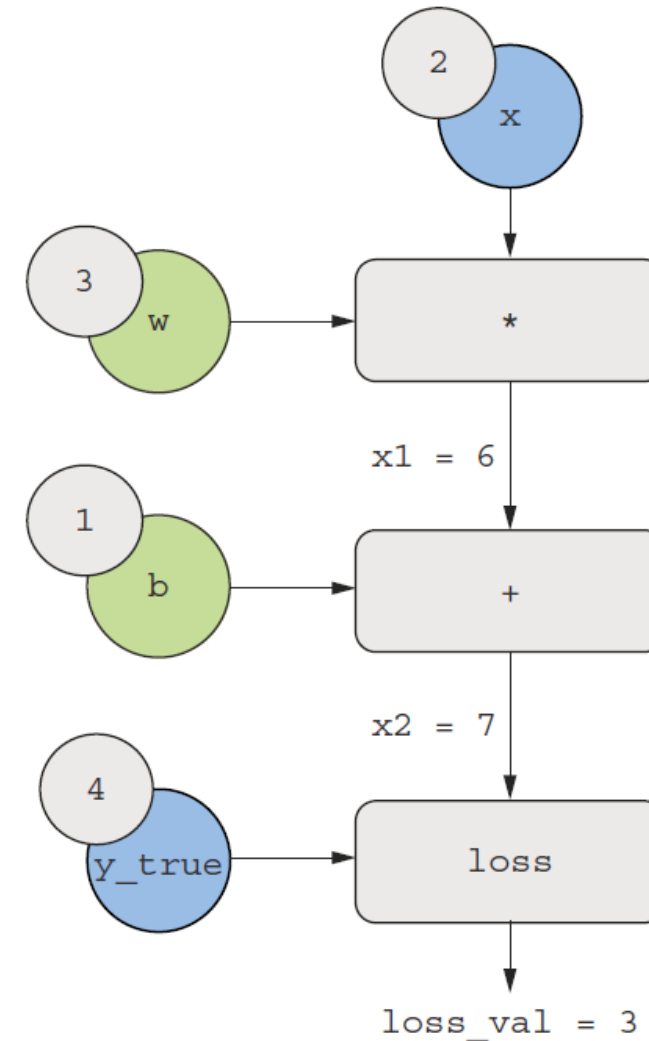
Vamos a asignar valores al dato de entrada x , el peso W y el bias b .

Sabemos, además, el valor real al cual debería llegar nuestra red (**y_{true}**).

Supongamos también, que las funciones de cada neurona son funciones simples (**multiplicaciones o sumas**).

Dado los valores entregados, el valor de pérdida encontrado es:

$$3 = 7 - 4.$$



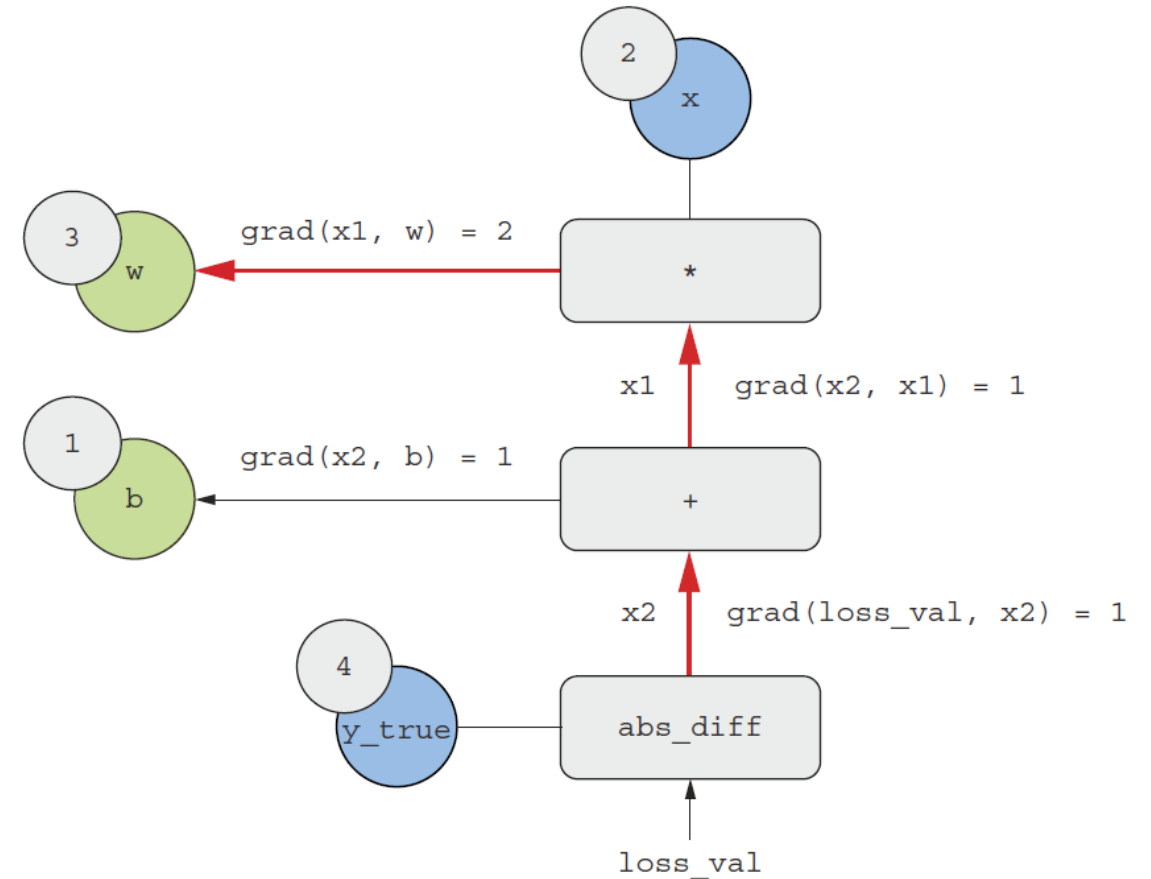
Ejemplo

Para encontrar la derivada de W sobre la función de pérdida, **Backpropagation**, **vuelve por la neurona hacia atrás**, calculando las derivadas parciales en cada punto.

Así supongamos los siguientes resultados (dependerán de la función utilizada en la neurona):

- La derivada de 'función de pérdida' y x_2 es = 1
- La derivada de x_2 y x_1 es = 1
- La derivada de x_2 y b es = 1
- La derivada de x_1 y w es = 2

Nota: derivada=grad



Ejemplo

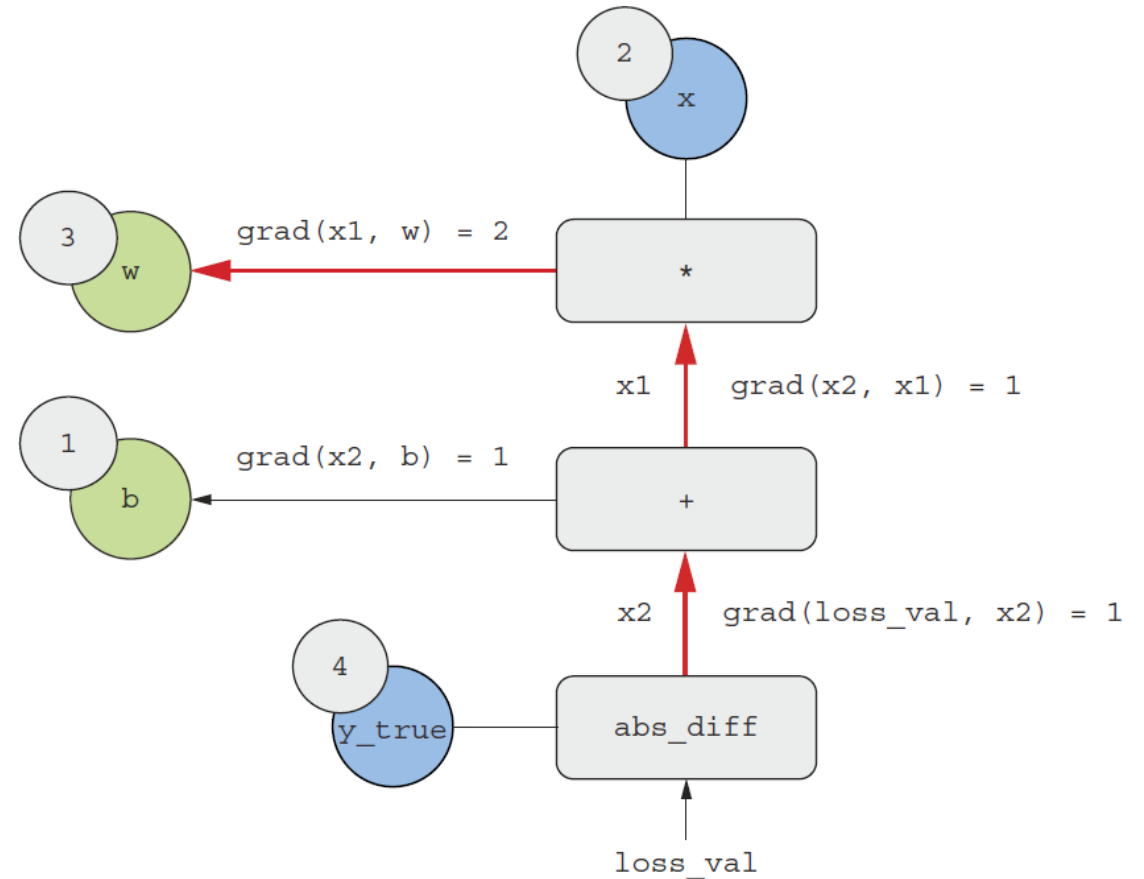
Luego, **Backpropagation**, aplicando la **regla de cadena** encuentra que :

- La derivada de **función de pérdida y W**
es $= 1 * 1 * 2 = 2$
- La derivada de la **función de pérdida y b**
es $= 1 * 1 = 1$

Así, el nuevo W y el nuevo b serán

$$W = W_0 - \text{Tasa de aprendizaje} * 2$$

$$b = b_0 - \text{Tasa de aprendizaje} * 1$$



04

Conclusiones



• Conclusiones

Sin entrar más profundamente en las matemáticas, el hecho es que, **aplicando la regla de la cadena, podemos ir de atrás hacia delante** calculando las derivadas parciales de la función de coste con respecto a todos los pesos y los bias.

A medida que vamos calculando derivadas parciales, las de la capa oculta a calcular a continuación (capa más próxima a la entrada de la red pues recordemos que estamos yendo de atrás adelante) hará uso de las derivadas parciales ya calculadas, por lo que el cálculo del gradiente resulta relativamente ... **sencillo**.

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO



NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la calidad / Vinculación con el Medio / Investigación, creación y/o innovación